

AUDITORIA TÉCNICA · NÃO É PARECER JURÍDICO

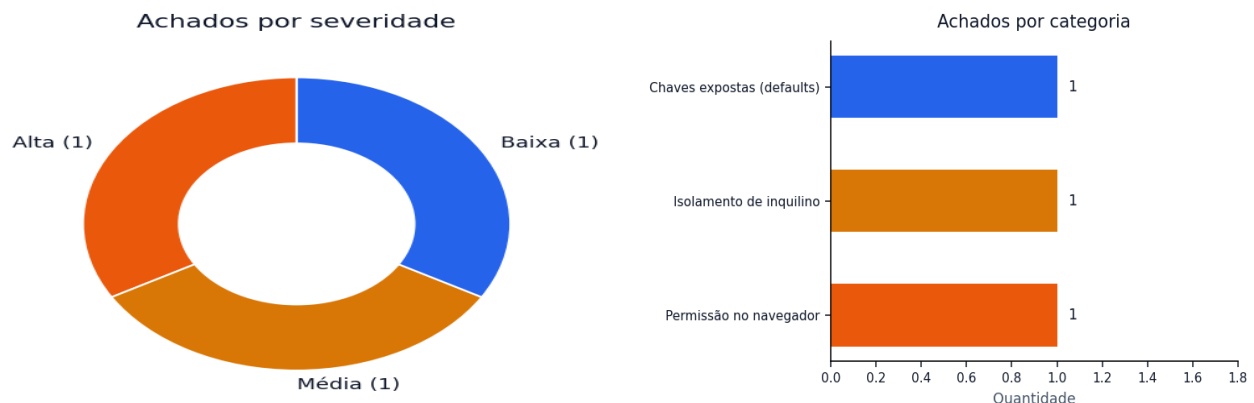
Relatório de Auditoria de Segurança — Prospectly

Data: 2026-08-29 **Escopo:** código em apps/api, apps/web, apps/landing, Prisma/migrations, Docker, Render Blueprint, GitHub Actions, histórico git.

Nota metodológica. Stack detectada: monorepo pnpm, API NestJS + Passport JWT, Prisma/PostgreSQL com RLS FORCE e guard de tenant em ALS, filas BullMQ/Redis, SPA React/Vite, landing Next.js, auth Argon2 + refresh HttpOnly. As cinco categorias foram traduzidas assim: (1) isolamento = RLS + interceptor + filtro organizationId do JWT — equivalente a “banco sem tranca”; (2) RBAC no servidor vs canManageOrg no React; (3) IDOR = todos os *.controller.ts da API (22 ficheiros), não amostra; (4) segredos em código, compose, .env.example, CI e git log -S; (5) XSS = dangerouslySetInnerHTML/href/src e HTML de e-mail. Só entram achados reproduzíveis no código. Ausência de categoria é reportada, não inventada.

1. Resumo executivo

Total de achados exploráveis: **3** (crítica 0, alta 1, média 1, baixa 1). IDOR entre tenants: **nenhum** verificado. XSS armazenado/refletido: **nenhum** verificado. Segredo de produção commitado: **nenhum**.



2. Pontos fortes

■ Isolamento multi-tenant em três camadas

CurrentOrg lê só request.user.organizationId (JWT). Prisma assertTenantOperation injeta/valida organizationId. Postgres FORCE ROW LEVEL SECURITY + role prospectly_app sem BYPASSRLS (DATABASE_APP_URL obrigatório em production).

■ IDOR por ID: handlers percorridos

Os 22 controllers da API usam CurrentOrg + findFirst({ id, organizationId }) (ou equivalente: pipeline.organizationId, importId após requireImport, candidateld+runId após requireRun). switchOrganization exige membership userId+organizationId. DSR e membros usam id + organizationId.

■ JWT e papéis nas escritas privilegiadas

APP_GUARD JwtAuthGuard + RolesGuard. Invite/role/remove member, billing checkout/profile/cancel, plugin tokens, scoring PATCH, export CSV e mutações de leads têm @Roles alinhado à UI canManageOrg.

■ Segredos de runtime

JWT via getOrThrow + validateEnv (mín. 32 chars; placeholder bloqueado em prod). Histórico git: ocorrências de AKIA só no regex do secret-scan. Sem sk_live_/chaves privadas commitadas. Plugin token armazenado como hash SHA-256.

■ XSS

Sem innerHTML de dado de usuário. JSON-LD estático com escape de <. E-mails com escapeHtml. Preview de campanha em . sanitizeAvatarSrc / sanitizeExternalUrl / sanitizeMailtoHref. wa.me só com dígitos (replace /D/g). Sem lib markdown.

3. Pontos fracos (risco residual)

- RolesGuard é fail-open: handler sem @Roles = qualquer membro autenticado, inclusive VIEWER. Isso é coerente com leituras de CRM, mas furado em GET /integrations.
- \$queryRaw/\$executeRaw no Prisma extension não aplicam GUC de RLS (tenant-prisma.ts:68–70). O gráfico de score do dashboard filtra organizationId no SQL. Em DATABASE_APP_URL a RLS sem GUC falha fechada. Risco residual se alguém adicionar raw SQL sem filtro no role owner.
- Rotas @Public() rodam runWithBypass. Correto para login/webhook/waitlist; qualquer query de tabela tenant nesses handlers perderia a rede RLS.
- Secret do webhook Abacate aceito também na query string (vazamento em access logs/proxies).

4. Tabela de achados

Sev.	ID / ficheiro:linha	Descrição
ALTA	AUD-001 apps/api/src/modules/integrations/integrations.controller.ts:22–25	GET /integrations lista webhook sem @Roles A UI esconde Integrações para quem não é OWNER/ADMIN (canManageOrg + adminOnly), mas GET /api/v1/integrations só exige JWT. RolesGuard libera a rota quando não há @Roles. A resposta inclui a URL completa do webhook (config.url).
MÉDIA	AUD-002 apps/api/src/modules/ops/ops-metrics.controller.ts:35–49	Métricas de ops são da instância, não do tenant GET /api/v1/ops/metrics exige OWNER/ADMIN da organização do JWT, mas devolve filas BullMQ compartilhadas, HTTP agregado do processo, pid, uptime e memória. Não há filtro por organizationId.
BAIXA	AUD-003 docker-compose.yml:7–10, 49	Senha default do Postgres no compose local POSTGRES_PASSWORD:-prospectly (e o mesmo valor em DATABASE_URL do serviço api). Portas bound a 127.0.0.1. Comentário avisa uso só local.

5. Achados detalhados

AUD-001 — GET /integrations lista webhook sem @Roles

Severidade: **Alta** · Categoria: Permissão no navegador ·
apps/api/src/modules/integrations/integrations.controller.ts:22–25

Problema. A UI esconde Integrações para quem não é OWNER/ADMIN (canManageOrg + adminOnly), mas GET /api/v1/integrations só exige JWT. RolesGuard libera a rota quando não há @Roles. A resposta inclui a URL completa do webhook (config.url).

Por que é explorável. VIEWER, MEMBER ou SALES autenticado chama GET /integrations e lê a URL do webhook da organização — inclusive se o admin colocou token na query. POST/DELETE de plugin e upsert do webhook estão protegidos com @Roles.

Impacto. Leitura de URL de integração (possível segredo no path/query). Não é escape entre tenants.

Trecho:

```
@Get()
list(@CurrentOrg() organizationId: string) {
  return this.integrations.list(organizationId);
}
```

Cruzamento: apps/web/src/features/settings/can-manage-org.ts:3–4; apps/web/src/features/settings/settings-shell.tsx:39–41; apps/web/src/pages/settings-page.tsx:72,75; apps/web/src/features/settings/integrations-settings-card.tsx:74; apps/api/src/common/guards/roles.guard.ts:34–36; apps/api/src/modules/integrations/integrations.service.ts:18–24,86–90

Correção. Anotar @Roles('OWNER', 'ADMIN') em GET /integrations, igual ao POST webhook.

AUD-002 — Métricas de ops são da instância, não do tenant

Severidade: **Média** · Categoria: Isolamento de inquilino · apps/api/src/modules/ops/ops-metrics.controller.ts:35–49

Problema. GET /api/v1/ops/metrics exige OWNER/ADMIN da organização do JWT, mas devolve filas BullMQ compartilhadas, HTTP agregado do processo, pid, uptime e memória. Não há filtro por organizationId.

Por que é explorável. Qualquer OWNER/ADMIN de qualquer tenant autenticado lê telemetria da plataforma (profundidade de filas globais, erros 5xx da instância). Não devolve payloads de jobs nem PII de outro tenant.

Impacto. Reconhecimento de infra e volume global. Sem leitura de CRM alheio.

Trecho:

```
@Get()
@Roles('OWNER', 'ADMIN')
async getMetrics() {
  const queues = await this.collectQueueDepths();
  ...
  return { process, http, jobs, queues, redis };
}
```

Cruzamento: apps/api/src/modules/ops/metrics.service.ts:108–119; apps/api/src/modules/ops/ops-metrics.controller.ts:66–98

Correção. Restringir a um papel de plataforma, IP allowlist, ou métricas agregadas só do tenant. Não expor pid/memória a clientes.

AUD-003 — Senha default do Postgres no compose local

Severidade: **Baixa** · Categoria: Chaves expostas (defaults) · `docker-compose.yml`: 7–10, 49

Problema. `POSTGRES_PASSWORD:-prospectly` (e o mesmo valor em `DATABASE_URL` do serviço `api`). Portas bound a 127.0.0.1. Comentário avisa uso só local.

Por que é explorável. Explorável só se o compose for publicado em host compartilhado sem sobrescrever o `env`. Produção no Render não usa esse default; `validateEnv` exige JWT e `DATABASE_APP_URL` em `production/staging`.

Impacto. Acesso local ao banco se a porta vazar ou o default for copiado para ambiente compartilhado.

Trecho:

```
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-prospectly}
DATABASE_URL: postgresql://${POSTGRES_USER:-prospectly}:${POSTGRES_PASSWORD:-prospectly}@postgres:5432/...
```

Cruzamento: `.env.example`: 6–8; `apps/api/src/config/validation.ts`: 4, 14–27, 42–49

Correção. Falhar o compose se `POSTGRES_PASSWORD` estiver vazio ou igual a `prospectly` fora de um perfil explicitamente local.

6. Cobertura IDOR (todos os controllers)

Percorridos os 22 `*.controller.ts` em `apps/api`. Nenhum handler de recurso por UUID foi encontrado sem amarrar o objeto ao tenant do JWT (ou, em privacidade, ao `user.id` do token).

- `auth.controller` — `switchOrganization` valida `membership`; rotas públicas são `auth`.
- `users.controller` — `/me` pelo JWT; DSR por `organizationId` + `@Roles OWNER`.
- `organizations.controller` — `memberId` + `organizationId`; `@Roles OWNER/ADMIN` nas escritas.
- `leads.controller` — `get/update/delete` com `organizationId`; `export` com `@Roles`.
- `prospecting.controller` — `requireSearch(organizationId, id)`.
- `campaigns.controller` — `requireCampaign`; `addLeads` recusa `leadIds` de outra org.
- `templates.controller` — `get(organizationId, id)`.
- `pipelines.controller` — `stage` via `pipeline.organizationId`.
- `tasks.controller` — `findFirst id+organizationId`.
- `activities.controller` — `assertLead(organizationId, leadId)`.
- `imports.controller` — `requireImport`; erros só após o `import` do tenant.
- `opportunity-finder.controller` — `requireRun`; `candidate id+runId`.
- `agents.controller` — `loadLead(organizationId, leadId)`.
- `billing.controller` — `CurrentOrg`; `checkout/profile` com `@Roles`; `webhooks @Public` com `secret`.
- `dashboard.controller` — `CurrentOrg` em `summary/charts`.
- `privacy.controller` — `user.id` do JWT (`export/erasure` do próprio titular).
- `scoring.controller` — `PATCH @Roles OWNER/ADMIN`.
- `geo.controller` — dados de referência, JWT, sem recurso por `id` de tenant.
- `waitlist/health` — públicos, sem objeto de tenant.

7. Categorias sem achado explorável

IDOR (cat. 3). Sem caso em que o ID de outro tenant devolva 200. Leituras de CRM sem `@Roles (VIEWER vê leads)` são papel de produto, não IDOR: o frontend também expõe essas páginas a qualquer membro autenticado.

XSS (cat. 5). `Landing` usa `dangerouslySetInnerHTML` só em JSON-LD estático com `<` escapado (`apps/landing/src/app/layout.tsx`: 87–90). Não há renderização de markdown de utilizador. HTML de e-mail de `reset` e `waitlist` passa por `escapeHtml`.

Chaves hardcoded de produção (cat. 4). Não foram encontradas API keys, JWT ou chaves privadas reais no tree. A constante HMAC da AbacatePay é a chave *pública* documentada pelo fornecedor; o segredo do webhook continua a ser ABACATE_WEBHOOK_SECRET. Client ID Google no bundle Vite é identificador OAuth público, não um segredo.

8. Recomendações priorizadas

- P1.** @Roles('OWNER','ADMIN') em GET /integrations + teste de regressão VIEWER.
- P2.** Isolar /ops/metrics (papel de plataforma ou remover da API de clientes).
- P3.** Recusar senha default do Postgres fora do perfil local.
- P4.** Aplicar GUC de RLS também em \$queryRaw (fail-closed já existe no role app).
- P5.** Preferir header ao secret de webhook Abacate na query string.

ISSUES PARA O GITHUB

Copiar cada bloco entre os delimitadores para uma issue nova. Labels sugeridas em cada bloco.

Issue 1

```

--- ISSUE 1 ---
# [Segurança] GET /integrations sem verificação de papel no servidor

Labels sugeridas: `security, alta`

## Problema
A aba Integrações some na UI para papéis que não são OWNER/ADMIN (`canManageOrg`), mas `GET /api/v1/integrations` não tem `@Roles`. O `RolesGuard` retorna `true` quando o metadata de papéis está vazio. A serialização devolve `url` do webhook.

## Por que é explorável
Um utilizador VIEWER/MEMBER/SALES com JWT válido (convite real) chama o endpoint e lê a URL de webhook da organização. Se o admin colou um token na query da URL, esse segredo vaza. Não atravessa tenant: `CurrentOrg` continua a vir do JWT.

## Evidência
```ts
// apps/api/src/modules/integrations/integrations.controller.ts:22-25
@Get()
list(@CurrentOrg() organizationId: string) {
 return this.integrations.list(organizationId);
}
...

```ts
// apps/api/src/common/guards/roles.guard.ts:34-36
if (!requiredRoles || requiredRoles.length === 0) {
  return true;
}
...

Cruzamento UI: `can-manage-org.ts:3-4`, `settings-shell.tsx:39-41` (`adminOnly`),
`integrations-settings-card.tsx:74` (`if (!canManage) return null`).

POST `/integrations/webhook` e tokens de plugin **já** usam `@Roles('OWNER', 'ADMIN')`.

## Impacto
Leitura não autorizada da URL de integração (e de qualquer segredo nela). Sem IDOR entre organizações.

## Correção sugerida
Adicionar `@Roles('OWNER', 'ADMIN')` em `GET /integrations`. Teste: VIEWER → 403; OWNER do mesmo tenant → 200; token do tenant B → 404/lista vazia.

## Critérios de aceite
- [ ] VIEWER recebe 403 em `GET /api/v1/integrations`
- [ ] OWNER/ADMIN lista só o webhook do próprio `organizationId`
- [ ] Regressão: upsert webhook e plugin tokens continuam 403 para MEMBER
--- FIM ISSUE 1 ---

```

Issue 2

```

--- ISSUE 2 ---
# [Segurança] /ops/metrics expõe telemetria da instância a qualquer admin de tenant

Labels sugeridas: `security, media`

## Problema
`GET /api/v1/ops/metrics` está autenticado com `@Roles('OWNER', 'ADMIN')` **do tenant do JWT**, mas o payload é da instância inteira: `Queue.getJobCounts` das filas globais BullMQ, snapshot HTTP do processo, pid, RSS/heap, ping Redis.

## Por que é explorável
Qualquer OWNER/ADMIN de um tenant cliente (não um operador de plataforma) obtém volume de filas de **todos** os tenants que partilham o processo. Não há `organizationId` nas contagens.

## Evidência
```ts
// apps/api/src/modules/ops/ops-metrics.controller.ts:35-48
@Get()

```

```

@Roles('OWNER', 'ADMIN')
async getMetrics() {
 const queues = await this.collectQueueDepths();
 ...
 return { process, http, jobs, queues, redis };
}
...

`getProcessSnapshot()` em `metrics.service.ts:108-119` inclui pid e memória.

Impacto
Reconhecimento de capacidade e carga da plataforma. Sem PII de outro tenant nas contagens.

Correção sugerida
1. Papel/segreto de plataforma (não role de tenant), ou
2. Não expor este controller na API pública, ou
3. Métricas só do tenant (jobs com `organizationId` no payload, sem pid).

Critérios de aceite
- [] OWNER de tenant de cliente não lê profundidade global das filas
- [] Operação interna ainda consegue health da API (`/health/ready` permanece público e mínimo)
--- FIM ISSUE 2 ---

```

## Issue 3

```

--- ISSUE 3 ---
[Segurança] Default prospectly/prospectly no docker-compose local

Labels sugeridas: `security, baixa`

Problema
`docker-compose.yml` usa `${POSTGRES_PASSWORD:-prospectly}` e monta `DATABASE_URL` com o mesmo
default. Portas em `127.0.0.1`.

Por que é explorável
Só se o compose for usado em host compartilhado/remoto sem sobrescrever a senha. Não é o
caminho Render (secrets no dashboard). `validateEnv` já rejeita JWT fraco e exige
`DATABASE_APP_URL` em production.

Evidência
```yaml
# docker-compose.yml:7-10
POSTGRES_USER: ${POSTGRES_USER:-prospectly}
POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-prospectly}
```

Impacto
Acesso ao Postgres de desenvolvimento se o default vazar para um ambiente não local.

Correção sugerida
Exigir `POSTGRES_PASSWORD` explícito, ou recusar o valor `prospectly` fora de um profile
`local`.

Critérios de aceite
- [] Compose local documentado com senha explícita
- [] Default `prospectly` não é usado em ambiente compartilhado
--- FIM ISSUE 3 ---

```